

Azure Console — Status Report & Close-the-Gaps Roadmap

As of 2026-05-28. Live at [/console/azure](#).

The current console is **functional but high-level** — it covers CRUD across 9 services with real SDK conformance and host-backed VM containers, but the user journey is a single-form blade per resource, not the multi-step wizard + sub-tab navigation real Azure portal users expect. This document records what's done and lays out a phased plan to close that gap so testing can begin.

Part 1 — Implementation status

1.1 The 9 services + lifecycle matrix

Service	ARM type	api-version	List	Get	Create	Update	Delete	Children	LRO	Data plane
Virtual Machines	Microsoft.Compute/virtualMachines	2023-09-01	✓	✓	✓	✓	✓	—	✓	Real LXD/multipass + host-budget gate + tier-clamped sizing
Storage accounts	Microsoft.Storage/storageAccounts	2023-01-01	✓	✓	✓	✓	✓	—	✓	Real blob bytes (sim-native REST, SharedKey-over-HTTP)
SQL servers	Microsoft.Sql/servers	2023-05-01-preview	✓	✓	✓	✓	✓	databases	✓	Real Postgres DBs per database, with connection info
Service Bus	Microsoft.ServiceBus/namespaces	2022-10-01-preview	✓	✓	✓	✓	✓	queues, topics	✓	REST send/receive broker (in-proc)
Cosmos DB	Microsoft.DocumentDB/databaseAccounts	2024-05-15	✓	✓	✓	✓	✓	sqlDatabases	✓	Sim-native SQL REST (dbs/colls/docs)
Functions	Microsoft.Web/sites	2023-12-01	✓	✓	✓	✓	✓	—	✓	—
API Management	Microsoft.ApiManagement/service	2023-05-01-preview	✓	✓	✓	✓	✓	—	✓	—
Virtual Network	Microsoft.Network/virtualNetworks	2023-11-01	✓	✓	✓	✓	✓	subnets	✓	—
Entra ID / RBAC	Microsoft.Authorization/roleAssignments	2022-04-01	✓	✓	✓	✓	✓	—	sync	—

1.2 Console UI surfaces — what's there today

Surface	Status	Notes
Top bar (Azure cube logo)	✓	Sticky #0b1b2b dark header
Subscription chip	✓	Shows active RG
Host-budget chip (Sim: X/Y CPU · X/Y MB)	✓	Live, color-tone, polled every 10s
Search box	△ placeholder	Renders only
Left service rail (Home + RGs + 9 services)	✓	Collapsible

Surface	Status	Notes
Home dashboard (per-service count cards)	✓	Live counts
Resource groups view	✓	
Service grid (per service)	✓	Columns from catalog, status badges
Toolbar (+ Create, ↺ Refresh)	✓	
Row-level Delete	✓	
Right-side blade with Overview/JSON/children tabs	✓	Slides in
VM runtime block (status, container, sizing, ⚠ host-clamped)	✓	Red row when clamped
Create blade (form built from catalog <code>createFields</code>)	✓	Single form per service
<code>vmSize</code> dropdown from instance catalog	✓	Grouped by family
Toasts (success/error)	✓	

1.3 Backend depth (verified)

- **Generic ARM dispatcher** parses any ARM URL (sub/RG/provider/type/name/childtype/childname) — nesting arbitrary.
- **POST actions** (`listKeys`, `regenerateKey`, `listConnectionStrings`, `listSecrets`).
- **LRO** with `Azure-AsyncOperation` + `Location` headers; status endpoint at `/api/azure/operations/{opId}`.
- **Provider registration** GETs (`/providers`, `/providers/{ns}`), POST `/register`).
- **Per-tenant ARM state** via `azure_arm_state` (`_SpaceScopedDictProxy`).
- **Provider lock per space** (Azure ops in non-Azure space → proxy scratch).
- **Data planes**: Blob (real bytes), SQL (real Postgres), Service Bus (REST), Cosmos (REST).
- **VM runtime backing**: `provision_azure_vm_runtime` → `_start_lxd_instance` / `_start_multipass_instance` with `runtime_sizing` (tier-clamped, host-aware).
- **Host budget gate**: 30%-50% clamp; pre-launch HTTP 403 with clear message.
- **Cloudsim graph** via `_cloudsim_collect_resources` (canonical path, not injection).

1.4 Real-SDK conformance (verified)

Client	Tests	Pass
Go (<code>azure-sdk-for-go</code>)	14	14
Java (<code>azure-resourcemanager-storage</code>)	5	5
az CLI	doc'd	— (usage docs at <code>/docs/azure/usage</code>)

1.5 Docs

Surface	URL
Swagger (synthetic, 29 paths, 10 tags)	<code>/docs/azure</code>
SDK & CLI usage docs	<code>/docs/azure/usage</code>
OpenAPI spec	<code>/openapi-azure.json</code>
Console	<code>/console/azure</code>
Catalog API	<code>/api/azure/catalog</code>
Budget API	<code>/api/runtime/budget</code>

1.6 Cross-cutting properties (structurally enforced)

- Cross-tenant access → **impossible** (state proxy).
- Cross-provider access in a space → **impossible** (Space 1:1 Provider).
- Host overload → **impossible** (budget gate clamp 30-50%).
- Stale capacity numbers → **impossible** (every mutation triggers refresh).

Part 2 — Where the console falls short of real Azure portal

The current console is **resource-CRUD-first**. Real Azure portal has a much richer journey:

Azure portal pattern	What we have	What we're missing
Create wizard with tabbed steps (Basics → Disks → Networking → Management → Advanced → Tags → Review+create)	Single-form blade	Multi-step wizard, validation, summary review
Resource detail with left sub-nav (Overview, Activity log, Access control, Tags, Settings → Networking, Disks, Size, Identity, Boot diagnostics ...)	Single blade with Overview + JSON + children tabs	Sub-nav, per-section pages, settings sub-tree
VM lifecycle commands (Start, Stop, Restart, Connect, Reset password, Capture, Auto-shutdown)	Create + Delete only	start/stop/restart actions, Connect button
Grids with column-toggle, multi-select, bulk actions, sort, filter, search	Static columns	Filter chips, sort headers, bulk select, search
Tag editing as a first-class tab with key/value editor	Tags via API only	Tag editor UI
Activity log per resource showing all operations	None	Operations history per resource
Cost analysis tab	None	Cost-per-resource + per-tenant
Diagnose & solve problems	None	Self-check pages
Monitoring tab (Metrics, Alerts, Logs)	None	Real-time metrics from the LXD container
Networking sub-tab on VM (NIC, NSG, public IP, DNS)	Not exposed	NIC/NSG/IP editing
Disks sub-tab on VM (OS disk, data disks attach/detach)	None	Disk management
Connect blade (RDP/SSH/Bastion options)	None	SSH/Bastion connection helpers
Cloud Shell terminal	Icon only	Working terminal into the container
Notifications bell with operation history	None	Toast history panel
Search resources (global)	Placeholder	Resource search across services
Pin to dashboard + dashboard view	None	Pinning, dashboard tiles
Subscription/RG picker in top bar	Static	Dropdown switcher

Part 3 — Close-the-gaps roadmap

Five phases, each independently shippable + testable. Targets 70% Azure-portal-likeness by end of Phase D (4 weeks of focused work). Phase E covers real-protocol heavy items (multi-week, can be parallel).

Phase A — VM lifecycle actions + console polish (2-3 days)

Goal: every Azure VM action a user expects on day-one works in the console.

Item	Backend	Console
Start/Stop/Restart/Deallocate for VMs	POST actions on <code>Microsoft.Compute/virtualMachines/{n}/start</code>	powerOff
Connect action	Endpoint that returns SSH cmd / <code>lxc exec</code> command (or opens a WebSocket console)	"Connect" button → modal with SSH/exec snippets
Tag editor	PATCH <code>tags</code> on any resource	Tags tab in detail blade with add/edit/delete key-value rows
Grid filter + sort + search	—	Column-header sort, free-text filter input above grid, multi-select chip filters per status/location
Refresh polling	—	Auto-refresh active grid every 15s
Notification panel (bell icon)	Reuse <code>_record_usage</code> events	Drawer listing recent actions across resources

Acceptance: create a VM → see it running → Stop → see status flip → Start → Connect → get SSH command → Delete; all from console without touching CLI.

Phase B — Multi-step Create wizards (3-4 days)

Goal: create flows look like Azure portal's tabbed wizard.

Per-service wizard tabs:

Service	Wizard tabs
VM	Basics (name, region, image, vmSize) · Disks (OS disk, data disks) · Networking (VNet, subnet, public IP, NSG) · Management (boot diagnostics, auto-shutdown) · Advanced (cloud-init) · Tags · Review+create
Storage	Basics (name, region, performance, redundancy) · Advanced (HNS, NFS, secure transfer) · Networking (public/private endpoint) · Data protection (versioning, soft delete) · Tags · Review+create
SQL Server	Basics (name, admin login, region) · Networking (firewall rules, public access) · Tags · Review+create
Service Bus	Basics (name, region, tier) · Tags · Review+create
Cosmos	Basics (account name, API, region, capacity mode) · Networking · Tags · Review+create
Function App	Basics (runtime, region) · Hosting (plan) · Networking · Tags · Review+create
APIM	Basics (name, region, tier, publisher email) · Networking · Tags · Review+create
VNet	Basics (name, region, IP addresses) · Subnets · Security (DDoS, firewall) · Tags · Review+create
RBAC role assignment	Role · Members · Conditions · Review+assign

Implementation: new `<WizardBlade>` component with tab nav, prev/next buttons, per-tab validation, Review step with diff table. Each tab is a small form component; the catalog's `createFields` extends to `wizardSteps[]`.

Acceptance: users navigate through tabs, hit Review, see all chosen values, hit Create → resource provisions correctly.

Phase C — Detail blade with left sub-nav (4-5 days)

Goal: detail blade matches Azure portal's left-nav structure.

For each service:

Resource: vm-web-01

Overview

Activity log

Access control

Tags

Diagnose+solve

— Settings —

Networking

Disks

Size

Properties

Locks

— Operations —

Auto-shutdown

Backup

Boot diagnostics

(current Overview tab content)

(events from _record_usage)

(IAM bindings for this resource)

(Phase A's tag editor)

(canned checks: budget? clamp?)

(NIC, NSG, IPs)

(OS + data)

(vmSize change → re-tier check)

(resourceId, type, sub, RG, etc.)

(CanNotDelete / ReadOnly)

(placeholder + planned)

(last 20 log lines from container)

Per-service sub-navs differ (Storage has "Containers", "File shares", "Access keys", "Networking"; SQL Server has "Databases", "Firewalls and virtual networks", "Active Directory admin"; etc.).

Implementation: rework `openDetail()` to render a 220-px left nav + main content area; each nav item renders a different React-less section. Sections share an Activity-log fetch + Tags editor.

Acceptance: the user can click into Networking on a VM, see VNet/subnet/NIC/NSG, change any, save → ARM PATCH applied.

Phase D — Per-service rich configurations (1-2 weeks)

Goal: each service exposes the configurations a real Azure user expects.

Service	Rich config
VM	NICs (create/delete), public IP attach, NSG rule editor, OS disk size, attach data disk, auto-shutdown schedule, boot diagnostics tail
Storage	Containers list + create + browse blobs + upload via UI · Access keys panel (regenerate) · Blob soft-delete · SAS token generator
SQL Server	Databases tab with create + delete · Firewall rules editor · Query editor (sends to backing Postgres) · Audit logs

Service	Rich config
Service Bus	Queue/topic create with maxSizeInMB, lockDuration, deadLetter, sessions · Send/receive playground panel
Cosmos DB	Database/container create with partitionKey, throughput (RU/s), indexing policy · Data Explorer (query playground)
Function App	Functions list (placeholder), Runtime config, App settings, Code & test (read-only)
APIM	APIs tree (placeholder) · Products · Subscriptions · Policy editor (XML)
VNet	Subnet drawer with delegations, service endpoints · Address space editor · Peerings
RBAC	Role assignment wizard with principal type (User/Group/SP) picker, role definition browser, conditions · Effective access view

Implementation: each service grows from "1 main grid" to "main grid + children grids + child-create wizards". Reuse the WizardBlade from Phase B for all child creates.

Acceptance: for at least Storage and VM, every config the user expects in Azure portal is editable in our console.

Phase E — Real-protocol completeness + polish (*parallel, multi-week*)

Item	Effort	Outcome
Cosmos SDK full protocol (resource tokens, partition keys, RU consumption)	large	<code>azure-cosmos</code> SDK works end-to-end against the sim
Service Bus AMQP via official emulator	large (+ disk for MS SQL Edge)	<code>azure-messaging-servicebus</code> SDK works
Azure CLI conformance harness	medium	<code>az smoke test</code> in <code>tests/conformance/azure-cli/</code>
Per-tenant SDK conformance	small	Re-run Go/Java harnesses with <code>X-Vyomi-Tenant</code> header set; verify isolation
Terraform <code>hashicorp/azurerm</code> round-trip gate	medium	Mirror the GCP terraform-gate
Cloud Shell real terminal in console (xterm.js + WebSocket to container)	medium	Click Cloud Shell icon → working bash in active VM
In-console tenant switcher chip	small	Top-bar dropdown like the SPA's TenantBar
Resource search (global)	small-medium	Search box matches across all services/tags
Dashboard / Pin to dashboard	medium	Pinned tiles view at <code>/console/azure/dashboard</code>
Cost analysis	medium	Per-resource + per-tenant cost from a pricing-per-vmSize sheet
Monitoring metrics	medium	CPU/memory from <code>lxc info</code> or <code>multipass info</code> → mini-graphs

Part 4 — Recommended order to start testing

1. **Today** — test what's there: confirm CRUD across all 9 services via the current console + verify host-budget gate, tenant isolation. Use the report's section 1 as a checklist.
2. **After Phase A** — test full VM journey including Start/Stop/Connect (~2-3 days from now).
3. **After Phase B** — test the wizard-style create flow on at least VM and Storage (~5-7 days).
4. **After Phase C** — test the rich detail blade with sub-nav (~10-12 days).
5. **After Phase D** — Azure-portal-equivalent feature coverage on the most-used services (~3-4 weeks).

Part 5 — File map (where the work lands)

Concern	File
Console UI	<code>static/azure-console.html</code>
Generic ARM dispatcher + LRO + POST actions	<code>providers/azure_services.py</code>
9-service catalog (extends to <code>wizardSteps[]</code> in Phase B)	<code>providers/azure_services.py</code> (<code>RESOURCE_CATALOG</code>)
Data planes (Blob, SQL, Service Bus, Cosmos)	<code>core/azure_dataplane.py</code>
Azure VM runtime provisioning	<code>server.py</code> (<code>provision_azure_vm_runtime</code>)

Concern	File
Instance-type catalog	<code>core/instance_catalog.py</code>
Host-aware tier sizing	<code>core/runtime_sizer.py</code>
Budget gate + tracker	<code>server.py (_simulator_budget , _check_budget_for_launch)</code>
Conformance harnesses	<code>tests/conformance/azure-sdk-{go,java}/</code>
Related memories	<code>azure-provider-build , tenant-isolation-model , heterogeneous-vm-shapes , instance-type-to-lxd-limits</code>

Bottom line for testing now

The Azure Console is **production-shape for CRUD + real protocols + host-aware runtime + structural isolation** — every claim in Part 1 is verified live. It is **prototype-shape for UI journey** — single-form blades, no wizards, no sub-nav, no Start/Stop, no tag editor.

If you want to test the *backend completeness* (LRO + data planes + real SDKs + isolation + budget): you can start today.

If you want to test the *Azure-portal-like user journey*: Phase A (2-3 days) brings the basics; Phase B+C (a week) makes the create + detail flow feel like Azure; Phase D (1-2 weeks) makes per-service config rich.

Recommended start: I begin Phase A immediately while you start testing what's there.